Etapa 4: Componente construite din date

Tehnologii Web · Proiect de semestru · Etapa 4

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Inginerie Industrială și Robotică
Disciplina: Tehnologii Web, proiect de semestru
Anul universitar: 2026–2027
Coordonatori de proiect: Emilia PĂSĂRIN, Alexandru RIZEA, Andrei-Alexandru STAICU

Enunțul etapei

Să se înlocuiască lista afișată ca JSON în Etapa 3 cu o listă de carduri desenată integral din date, prin două componente React: una pentru un singur element și una pentru întreaga listă. Datele se transmit componentelor prin **props**. Lista goală trebuie să afișeze un mesaj explicit, iar consola browserului nu trebuie să conțină avertismente.

Cardurile apar pentru că există elemente în array, nu pentru că au fost scrise manual în HTML: adăugarea unui element în array produce un card în plus, fără nicio altă modificare de cod. Cardul nu mai este cod repetat de trei ori, ci o singură componentă folosită de trei ori. Rezultatul vizual este identic cu mockup-ul din Etapa 1.

La finalul etapei veți putea:

- împărți interfața în componente, câte una pe fișier;
- transmite date între componente prin props;
- randa o listă din array cu `map` și atributul `key`;
- afișa conținut în funcție de o condiție (randare condiționată);
- interpreta avertismentele React din consola browserului.

Noțiuni introductive

Componente și props

O componentă este o funcție care întoarce JSX și al cărei nume începe cu literă mare. Fiecare componentă se scrie în fișierul propriu, în folderul `src/components/`, iar numele fișierului coincide cu numele componentei.

Props sunt datele transmise de componenta părinte către componenta copil. Se scriu ca atribute, iar copilul le primește ca obiect, de obicei destructurat direct în lista de parametri:

```
// componenta părinte transmite
<TaskCard task={task} />

// componenta copil primește
function TaskCard({ task }) {
  return <h3>{task.titlu}</h3>;
}
```

Props-urile pot fi doar citite: componenta copil nu le modifică. Modificarea datelor se face prin componenta părinte, mecanism introdus în Etapa 5. Figura 4.1 prezintă componentele din această etapă și datele transmise între ele.

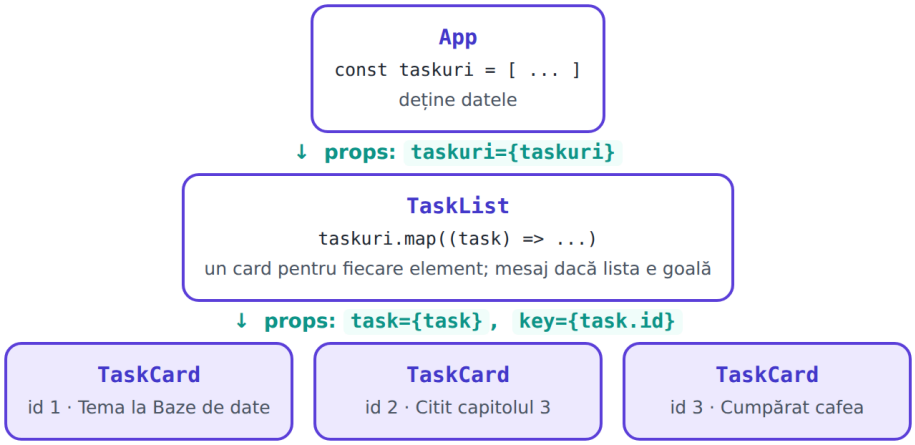


Fig. 4.1: Structura componentelor și datele transmise prin props

Randarea unei liste și atributul key

O listă se randează cu `map`, aceeași metodă din Etapa 2: pentru fiecare element din array se obține un element JSX. Fiecare element randat dintr-o listă are nevoie de atributul **key**, o valoare unică și stabilă după care React recunoaște elementele între două randări. `key` se pune pe componenta întoarsă direct din `map` și are ca valoare id-ul elementului, nu poziția din listă; folosirea poziției produce erori la ștergere, în Etapa 5.

Randarea condiționată

Două forme folosite frecvent în etapele următoare:

```
{task.gata ? "Gata" : "Activ"}                // una dintre două variante
{taskuri.length === 0 && <p>Listă goală</p>}  // un element sau nimic
```

Adaptarea la tema proprie

Tabelul 4.1: Denumirile pe tema proprie

Element	TaskFlow	BookShelf	WatchList	Tema proprie
componenta pentru un element	TaskCard	CarteCard	FilmCard	
componenta pentru listă	TaskList	CarteList	FilmList	
props-ul transmis de App	taskuri	carti	filme	
etichetele afișate	ETICHETE_PRIORITATE	ETICHETE_FORMAT	ETICHETE_TIP	
textul stării	Activ / Gata	Necitită / Citită	De văzut / Văzut	
mesajul pentru lista goală	Nu există niciun task.	Nu există nicio carte.	Lista de filme este goală.	

Câmpurile suplimentare ale temei (de exemplu autorul sau durata) se afișează tot în card, sub titlu.

Desfășurarea lucrării

Serverul de dezvoltare pornit cu `npm run dev` (Etapa 3) rămâne deschis pe toată durata lucrului; pagina se actualizează la fiecare salvare.

Pasul 1: Folderul de componente

În `frontend/src/` se creează folderul `components/`. Toate componentele se plasează aici, câte una pe fișier. La finalul etapei, structura este cea din figura 4.2.

```
student@pc:~$ cd proiect-tw/frontend
student@pc:~/proiect-tw/frontend$ tree src
src
├── App.css
├── App.jsx
├── components
│   ├── TaskCard.jsx
│   └── TaskList.jsx
└── main.jsx

2 directories, 5 files
student@pc:~/proiect-tw/frontend$
```

Fig. 4.2: Structura folderului src la finalul Etapei 4

Pasul 2: Componenta pentru un element (TaskCard)

Fișier nou: `frontend/src/components/TaskCard.jsx`. Componenta primește un singur element și afișează exact ce afișa un card scris manual în Etapa 1:

```jsx
// Un singur element din listă, primit prin props.
const ETICHETE_PRIORITATE = { mica: "mică", medie: "medie", mare: "mare" };

export default function TaskCard({ task }) {
  return (
    <li className={task.gata ? "task-card done" : "task-card"}>
      <div>
        <h3>{task.titlu}</h3>
        <span className={`badge badge-${task.prioritate}`}>
          Prioritate {ETICHETE_PRIORITATE[task.prioritate]}
        </span>
      </div>
      <span className="status">{task.gata ? "Gata" : "Activ"}</span>
    </li>
  );
}
```

Explicații:

- `{ task }` destructurează props-urile: componenta primește un obiect cu proprietatea `task` și o extrage direct într-o variabilă;
- `className={task.gata ? "task-card done" : "task-card"}` alege clasa în funcție de date, astfel încât elementul finalizat arată diferit, ca în mockup;
- `` `badge-${task.prioritate}` `` construiește numele clasei din valoare: pentru `mare` rezultă `badge-mare`, clasă definită deja în CSS în Etapa 1;
- `ETICHETE_PRIORITATE` traduce valoarea din date în textul afișat: în date rămâne `mica`, fără diacritice, iar pe ecran apare „mică”.

Pasul 3: Componenta pentru listă (TaskList)

Fișier nou: `frontend/src/components/TaskList.jsx`. Componenta primește întreaga listă și randează câte un card pentru fiecare element. Implementarea urmează structura de mai jos:

```jsx
import TaskCard from "./TaskCard.jsx";

export default function TaskList({ taskuri }) {
  // 1. dacă lista este goală:
  //    return <p className="empty">Nu există niciun task.</p>
  // 2. altfel: return <ul className="task-list"> ... </ul>, cu câte un
  //    <TaskCard key={task.id} task={task} /> pentru fiecare element (map)
}
```

Verificarea listei goale se face prima, printr-un `return` timpuriu: dacă nu există elemente, componenta se oprește acolo. Atributul `key` se pune pe `TaskCard`, adică pe elementul întors direct din `map`, nu pe un element din interiorul cardului.

Pasul 4: Utilizarea componentelor în App.jsx

În `App.jsx` se importă lista, în partea de sus a fișierului, iar blocul temporar `<pre>{JSON.stringify(...)}</pre>` din Etapa 3 se înlocuiește cu componenta:

```
import TaskList from "./components/TaskList.jsx";
...
<section className="panel">
  <h2>Task-urile mele</h2>
  <TaskList taskuri={taskuri} />
</section>
```

`taskuri={taskuri}` transmite props-ul numit `taskuri`, cu valoarea variabilei `taskuri`. Numele din stânga trebuie să fie identic cu cel destructurat în componentă; în caz contrar, copilul primește `undefined`. După salvare, pagina arată ca în figura 4.3.

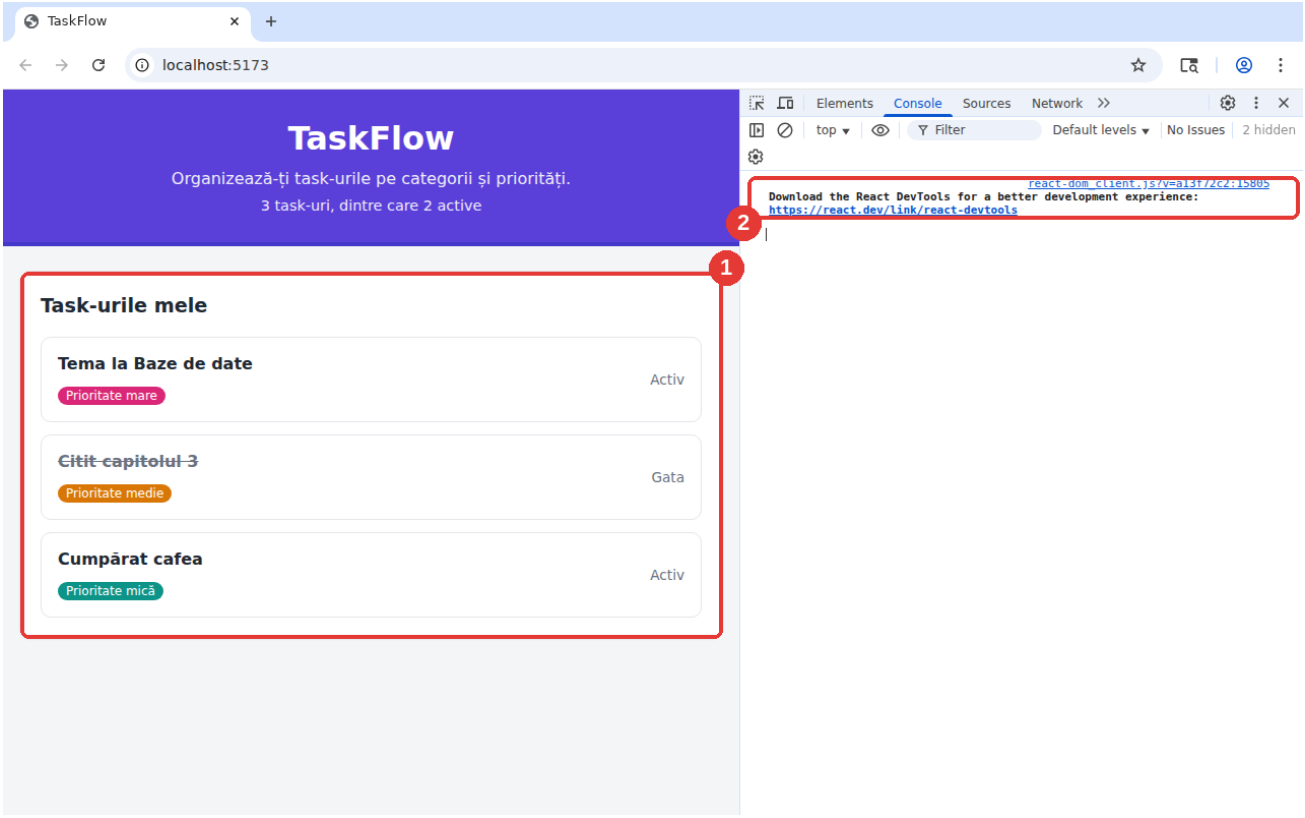


Fig. 4.3: Lista desenată din date, cu consola deschisă

1. Cardurile sunt generate de componenta `TaskList`, câte unul pentru fiecare element din array.
2. Consola conține doar mesajul informativ al React; nu apar avertismente sau erori.

Pasul 5: Testarea cazurilor speciale

Atributul key. Dacă `key` lipsește sau este pus pe un element greșit, React afișează în consolă avertismentul din figura 4.4. Pentru a-l recunoaște, se poate elimina temporar `key={task.id}`, se salvează, se observă avertismentul, apoi atributul se pune la loc.

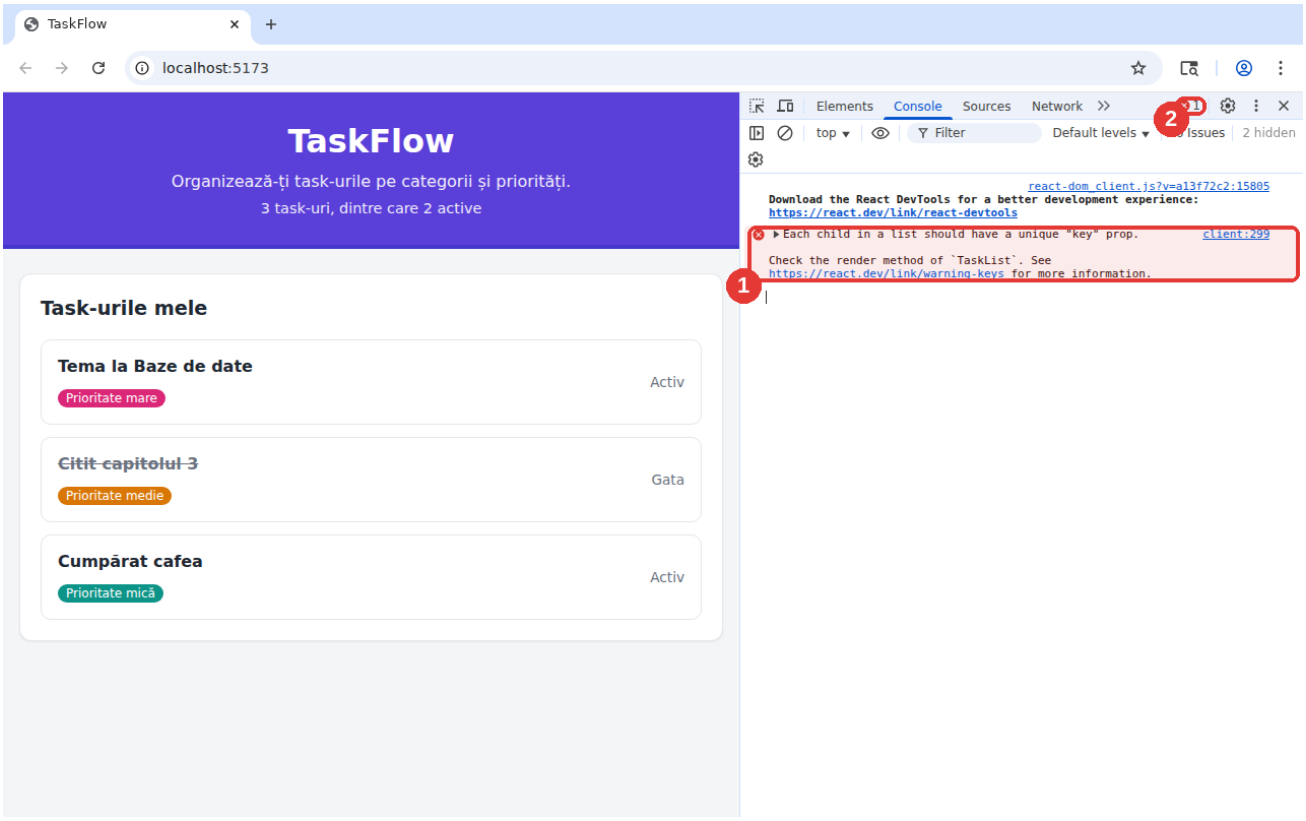


Fig. 4.4: Avertismentul afișat când lipsește atributul key

1. Mesajul *Each child in a list should have a unique “key” prop*, cu indicația componentei în care apare problema (`TaskList`).
2. Contorul de erori din DevTools.

Lista goală. Array-ul se înlocuiește temporar cu `[]` și se salvează. Trebuie să apară mesajul pentru lista goală, iar contorul din antet trebuie să afișeze zero (figura 4.5). Apoi datele se pun la loc.

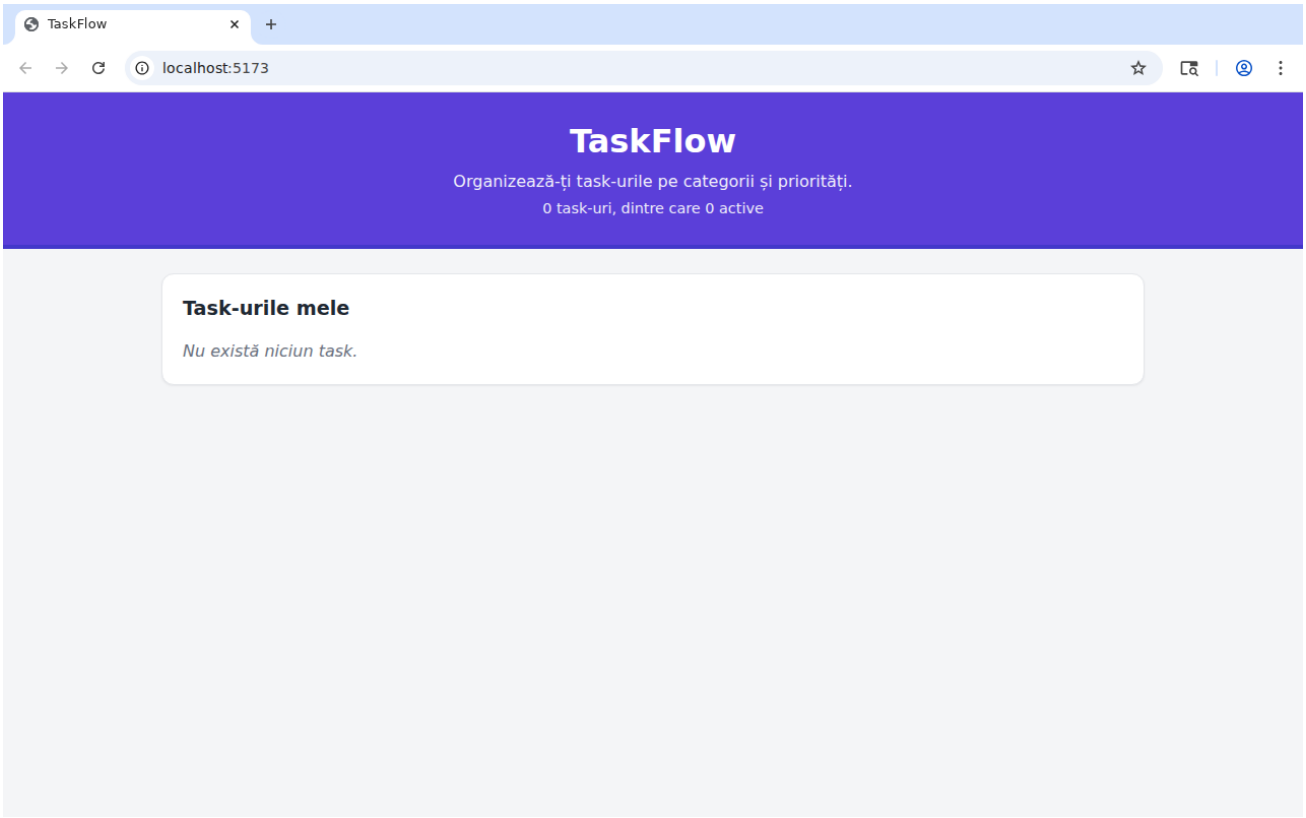


Fig. 4.5: Mesajul afișat pentru lista goală

Schimbarea stării. Se modifică valoarea `gata` la un element și se salvează. Cardul se schimbă imediat, fără altă modificare de cod.

Pasul 6: README, jurnalul AI și publicarea pe GitHub

În README se adaugă secțiunea etapei și se actualizează stadiul:

```
## Stage 4: components
The list is rendered from the data, through two components:
- `TaskList` receives the whole array and renders one card per item
- `TaskCard` receives a single item through props and displays it

## Status
- [x] Stage 3: Vite and React project
- [x] Stage 4: components rendered from data
□ Stage 5: local state and events
```

Se completează `ai-log/etapa-04.md`, apoi:

```
git add .
git commit -m "Stage 4: components rendered from data"
git push
```

Tabelul de verificare din README

După publicarea codului, în README se adaugă tabelul de verificare al etapei (modelul din Etapa 1), publicat într-un commit separat, `Stage 4: checklist`.

Tabelul 4.2: Tabelul de verificare al Etapei 4

ID	Requirement	Where (permalink)	How to check
S4-R1	2+ components in src/components/	frontend/src/components/	open folder
S4-R2	card receives data through props	TaskCard.jsx#L..-L..	read
S4-R3	map + key={id}, no console warnings	TaskList.jsx#L..-L..	F12, Console
S4-R4	message for empty list	TaskList.jsx#L..-L..	set array to []
S4-R5	state changes the card look	TaskCard.jsx#L..	toggle a done flag
S4-R6	README Stage 4 section + AI log	README.md, ai-log/etapa-04.md	read
S4-R7	commit “Stage 4” pushed	link to the commit	commit history

Lista de verificare a etapei

☐ **S4-R1** Există cel puțin două componente în `src/components/`, câte una pe fișier.

☐ **S4-R2** Cardul primește datele prin props și nu citește array-ul direct.

☐ **S4-R3** Lista este randată cu `map` și `key={id}`, fără avertismente în consolă.

☐ **S4-R4** Lista goală afișează un mesaj.

☐ **S4-R5** Starea elementului modifică aspectul cardului.

☐ **S4-R6** README-ul este actualizat, iar `ai-log/etapa-04.md` este completat.

☐ **S4-R7** Commit-ul `Stage 4: ...` este publicat pe GitHub.

Criterii de calitate urmărite: componentele primesc prin props tot ce afișează și pot fi reutilizate; `key` are ca valoare id-ul, nu poziția; etichetele afișate sunt separate de valorile din date; consola nu conține avertismente React.

Probleme frecvente

Tabelul 4.3: Probleme frecvente și rezolvarea lor

Simptom	Cauză probabilă	Rezolvare
avertisment despre `key` în consolă	`key` lipsește sau este pus în interiorul cardului	`key` pe componenta întoarsă direct din `map`
`task is undefined`	props transmis cu alt nume decât cel destructurat	`<TaskCard task={task} />` și `function TaskCard({ task })`
nu apare nimic, fără erori	`map` cu acolade, fără `return`	`(task) => (<TaskCard ... />)`, cu paranteze rotunde
`does not provide an export named 'default'`	fișierul componentei nu are `export default` sau nu este salvat	verificarea primului rând: `export default function ...`
`Element type is invalid`	import cu altă cale sau alt nume de fișier	verificarea căilor: `./components/TaskList.jsx` din App, `./TaskCard.jsx` din TaskList
pagină complet albă	React s-a oprit la randare	eroarea apare în consola browserului (F12, fila Console)
badge-ul nu are culoare	valoarea din date nu corespunde unei clase CSS	clasele CSS trebuie să acopere toate valorile posibile

Cerințe până la etapa următoare

Până la etapa următoare, componentele, README-ul actualizat (cu tabelul de verificare) și jurnalul `ai-log/etapa-04.md` trebuie să fie publicate pe GitHub. În Etapa 5, lista devine interactivă: elementele pot fi bifate și șterse din interfață, folosind funcțiile scrise în Etapa 2.